FACULTAD DE
INGENIERÍA
UNIVERSIDAD DE SANTIAGO DE CHILE

Paradigmas de Programación Laboratorio 3: Simulación de sistema de archivos en Java.

Nombre: Isidora Oyanedel

Profesor: Gonzalo Martínez

17 de Julio 2023

Índice

1. INTRODUCCIÓN **3**
1.1 DESCRIPCIÓN DEL PROBLEMA 3
1.2 DESCRIPCIÓN DEL PARADIGMA 3
1.3 OBJETIVOS 4
2. DESARROLLO **4**
2.1 ANÁLISIS DEL PROBLEMA 4
2.2 DISEÑO DE SOLUCIÓN 5
2.3 ASPECTOS DE IMPLEMENTACIÓN 5
2.4 INSTRUCCIONES DE USO 6
2.5 RESULTADOS Y AUTOEVALUACIÓN 6
3. CONCLUSIÓN **6**
4.BIBLIOGRAFÍA **7**
5.ANEXO **7**

1. INTRODUCCIÓN

El presente informe corresponde al laboratorio n°3 de Paradigmas de programación, donde nos centramos en el paradigma orientado a objetos y utilizamos el lenguaje de programación Java, a través de sus herramientas debemos llegar a la resolución del problema planteado, el cual es la creación de un simulador de sistema operativo, centrado en un sistema de archivos incluyendo comandos que permitan operar en él incluyendo la creación de un Menú interactivo y un diagrama UML que representa nuestro proyecto.

1.1 DESCRIPCIÓN DEL PROBLEMA

Como se mencionó, se pide desarrollar un simulador de sistema operativo en el cual se puedan crear drives, usuarios y poder crear carpetas y archivos dentro de los drives, los cuales tendrán sus respectivos permisos, especificaciones, entre otras funciones. Para lo cual se necesita crear ciertos elementos que sean capaces de hacerlo, como, por ejemplo:

Sistema: El elemento principal para iniciar la solución del problema, al tenerlo creado se puede empezar a crear los drives, usuarios, carpetas, rutas, etc. Pues en él serán almacenadas estas unidades.

Drive: Creaciones del usuario, en donde el mismo puede seleccionar alguno para poder trabajar en él y aplicarle cambios. Además se pueden realizar acciones entre drives a pesar de que no sea el seleccionado para trabajar.

Usuario: Son los usuarios registrados en el sistema, los cuales se identifican por un nombre de usuario. Se le atribuyen permisos, con los cuales, mientras esté logueado, será capaz de utilizar las funciones del sistema.

Archivos/carpetas: Creaciones del usuario, en cada creación se registra un autor, la fecha de creación, su nombre, su contenido, entre otros aspectos principales en el que cada cambio quedará registrado.

Otras operaciones: Funciones encargadas de loguear, desloguear, crear, modificar, en el sistema. En resumen, hacen el trabajo necesario.

Cada uno de los elementos especificados debe ser implementado en el lenguaje Java centrando su implementación en el paradigma Orientado a objetos.

1.2 DESCRIPCIÓN DEL PARADIGMA

En este laboratorio nos centramos en el uso del paradigma orientado a objetos, siendo paradigma una especie de patrón y paradigma funcional un estilo de programación que se basa en clases y objetos para resolver problemas.

En el paradigma existen ciertos conceptos que son claves al momento de trabajar con él:

Clases: Modelos que describen propiedades y comportamientos de un objeto.

Métodos: Funciones definidas dentro de una clase que describen los comportamientos de los objetos. Pueden realizar operaciones y manipular los objetos.

Atributos: Características que poseen los objetos y los permite diferenciar de otros.

Herencia: Proceso donde una clase puede tener propiedades o comportamientos de otras clases principalmente para reutilizar código.

Dependencia: Instancia en la que una clase depende de otra clase para poder llevar a cabo ciertas funciones propias.

Interfaz: Especificación de métodos que otras clases pueden implementar.

Agregación: Cuando una clase contiene una o varias instancias de otra clase.

Asociación: Cuando una clase está relacionada con otra, pero esta relación entre ambas no es una dependencia fuerte por lo que pueden vivir de manera independiente.

Composición: Cuando una clase tiene instancias de otra clase pero estas instancias no pueden existir de manera independiente a la clase que los contiene.

1.3 OBJETIVOS

En este proyecto se aprenderá el cómo funciona el paradigma orientado a objetos, también algunos aspectos nuevos de la programación en el lenguaje Java y sobre cómo entendíamos la programación antes de adentrarnos en este paradigma además de aprender a programar en lenguaje Java y entender las herramientas del mismo a través del desarrollo del laboratorio.

2. DESARROLLO

2.1 ANÁLISIS DEL PROBLEMA

Se desarrolla la simulación de un sistema de archivos, dando a entender que ninguno de los datos que se trabajan son reales y es una simulación simple que no busca ser 100% fiel a uno real y funcional. Partiendo por el elemento principal, el sistema, que es a partir del cual se pueden desarrollar el resto de funciones que hacen funcionar la simulación, y guarda todos los datos ingresados, como drives, usuarios, archivos, fechas, etc.

Las funciones que se pueden aplicar en el sistema son:

- add-drive: verifica si hay un drive con la misma letra del que se quiere crear, si no es así, se crea un nuevo drive.
- register: verifica si hay un usuario con el mismo "username", si no es así se crea un usuario nuevo.
- login: loguea un usuario si es que está registrado su username.
- logout: desloguea al usuario del sistema.
- switch-drive: fija la unidad de drive en la que el usuario trabajará.
- md (make directory): crea un directorio dentro de una unidad.
- cd (change directory): cambia la ruta donde se realizan las operaciones.
- add-file: agregar un archivo en la carpeta en la ruta.
- del: elimina un elemento o varios, puede ser un archivo o una carpeta.
- copy: copia una carpeta o archivo a una ruta de destino desde una ruta de origen.
- move: mueve un archivo o una carpeta.
- ren (rename): renombra una carpeta o un archivo.
- dir (directorio): muestra el contenido que hay en una registro o una ruta entera.
- format: formatea el contenido de un drive.
- encrypt: encripta un archivo o una carpeta.
- decrypt: desencripta un archivo o una carpeta.
- view-trash: obtiene el contenido que hay en la papelera de reciclaje.
- restore: recupera el contenido que se encuentra en la papelera.

Con la condición de que para poder acceder a las carpetas ya sea para agregar contenido o verlo, se necesita que haya un usuario con sesión iniciada.

2.2 DISEÑO DE SOLUCIÓN

Se ha necesitado la creación de diversas clases de datos para poder desarrollar el problema.

- CLASE SYSTEM: lista de listas que contiene el nombre del archivo, fecha, usuarios, drives, etc.
- CLASE DRIVE: Encargado de crear y cambiar de drive según lo solicite el sistema.
- CLASE USER : Lista que contiene a los usuarios correctamente creados.
- CLASE FOLDER lista de listas que contiene los archivos, la fecha de modificación, la fecha de creación y la ruta actual.
- CLASE FILE: lista que contiene el archivo y su contenido.

Al momentos de ir realizando el código hay detalles que deben ser tomados en cuenta:

- **Funciones map y filter:** Estás funciones vuelven a aparecer en Java, las cuales en su momento fueron muy útiles en Scheme por lo que ayudó a poder abordar el problema de una manera similar al laboratorio de Scheme porque al momento de ir avanzando en el código, se llega a un punto en el que agregar elemento no basta para solucionar los problemas que van naciendo por lo que se recurre a estas funciones ya que permiten obtener y aplicar operaciones sobre el sistema, es decir, al usar filter, permite buscar o aplicar cambios sobre una carpeta o archivo específico, en cambio map, permite aplicar funciones sobre un elemento, esto igual fue de gran utilidad al momento de querer hacer operaciones sobre una lista con varios elementos.
- **Completación de código de Java:** Al momento de empezar el laboratorio no se sabía muy bien cómo funcionaba Java y sus conceptos de herencia, dependencia, entre otros. Pero con esta herramienta sabes si vas haciendo bien las cosas o no porque ya al momento de escribir lo que necesitas y si te sale de sugerencia para colocarlo sabes que vas por buen camino, en caso contrario sabes que debes cambiar algo o que estás accediendo de mala manera a algún lugar.

2.3 ASPECTOS DE IMPLEMENTACIÓN

En este laboratorio, se utilizó el entorno de desarrollo IntelliJ IDEA (Community Edition) junto con su compilador correspondiente, que es la versión de Amazon Corretto 11 de Java. No se emplearon bibliotecas externas, cumpliendo con las exigencias del enunciado del laboratorio. Además, se realizaron diagramas UML, donde el primero fue una predicción del resultado del programa y el segundo representó el resultado final del programa con todos los conceptos del diagrama UML aplicados (figuras 4 y 5 en el anexo).

Para este laboratorio ya no existen los TDA sino que ahora nos adaptamos a las “clases” de Java, las cuales nos permiten aprovechar las herramientas que entrega Java y del mismo Paradigma orientado a objetos, además de que son más fáciles de modificar que un TDA. Pero a pesar de eso hay que admitir que un TDA era más simple, pues ahora las clases abren un mundo amplio de posibilidades con las herencias, dependencias, entre otros. que puede generar siendo estas herramientas unas de las más actuales de la programación moderna.

2.4 INSTRUCCIONES DE USO

Se necesita que se tengan descargados y juntos todos los archivos del proyecto, caso contrario el programa probablemente tirará error por falta de una clase o interfaz.
En este laboratorio no existe un script de pruebas, por lo tanto puede interactuar con el menú tanto como desee, pues todo se realizará según las elecciones que ha escogido. Las funciones disponibles para usar en el sistema son solo las que han sido colocadas en el menú, por lo que si faltan es porque no se han realizado a nivel de código. Además recordar que se necesita si o si un usuario logueado para ciertas funciones.

1. Iniciar el programa a través del archivo "MenuInteractivo_21168603_ioyanedelalvarez.bat" dentro de la carpeta "ProyectoCodigoFuente_Laboratorio3_21168603_OyanedelAlvarez" y colocar la opción 1 de creación de sistema. De igual forma, esto se puede ejecutar a través de la consola de IntelliJ IDEA a través del archivo Main.java.
2. Seleccionar una opción luego de seleccionar modificar sistema. Cada opción corresponde a una de las funciones del laboratorio, por ejemplo la opción 1 será crear drive y así sucesivamente.
3. Revisar los resultados en la consola con la opción 3.

Revisar figuras 1, 2 y 3 para una demostración gráfica del funcionamiento.

2.5 RESULTADOS Y AUTOEVALUACIÓN

RESULTADOS ESPERADOS:

Se esperan leves fallas en el código, más que nada en las funciones move y copy. Y a pesar de no haber scripts de prueba se espera que el usuario que manipule el programa siga las especificaciones, sino obviamente este va a fallar.

AUTOEVALUACIÓN:

0: No realizado – 0.25: Funciona 25% de las veces – 0.5: Funciona 50% de las veces 0.75: Funciona 75% de las veces – 1: Funciona 100% de las veces.

Ver tabla n° 1 y 2 de autoevaluación en el anexo.

La función copy tiene 0.75 pts. porque hay momentos en que no es capaz de copiar la/s carpeta/s en las carpetas del destino, ocurre lo mismo con la función move.

3. CONCLUSIÓN

Al haber realizado la mayoría de la actividad, se tuvo como resultado el haber podido completar los objetivos planteados al inicio, pues a pesar de que no se pudo crear gran cantidad de las funciones solicitadas del proyecto se ha podido aprender a manejar Java y una pequeña fracción de las herramientas que ofrece.

A pesar de que este es un laboratorio considerado corto por todas las facilidades que entrega Java si resultó complejo a la hora de entender lo que es una herencia, una dependencia o una composición de clases, por lo que algunos requerimientos fueron complejos de entender a la primera. Además la falta de tiempo por cierre de semestre fue un obstáculo importante para este laboratorio y el aprendizaje de la sintaxis tuvo que ser más rápido que en los laboratorios previos, pues tuvimos menos clases para aprenderlo por el mismo tema de cierre de semestre.

Ya al ser el tercer laboratorio uno ya se da cuenta de cómo ayudó la realizaciones de los anteriores para poder repensar en la forma de abordar los problemas, porque se vuelve a las herramientas que se nos fueron presentadas en los laboratorios anteriores, nos permitieron mejorar la forma de abordar estos problemas.

4.BIBLIOGRAFÍA

Martínez, G. (2023). “proyecto semestral de laboratorio”. Obtenido de la Universidad de Santiago de Chile. Paradigmas de Programación. Enunciado de Proyecto Online. Recuperado de: https://docs.google.com/document/d/1Wc-9_OFU7zsi5K74-opXv-cu7I8cCApUPxbvTkrRjCo/edit

Gacitúa, D. (2023). “5 - P. Orientado a Objetos”. Obtenido de la Universidad de Santiago de Chile. Paradigmas de Programación. Material de clases Online. Recuperado de: https://uvirtual.usach.cl/moodle/course/view.php?id=10036&section=20

5.ANEXO

Tabla 1: Auto Evaluación de requerimientos NO funcionales.

Nombre	Puntaje
Autoevaluación	1
Documentación	1
Versión	1
Interacción con el programa	1
Uso del Paradigma	1
Standard	1
Lenguaje	1
Historial	1
Prerrequisitos	1
Organización de código	1
Diagrama de análisis	1
Diagrama de diseño	1
Github	1

Tabla 2: Auto Evaluación de requerimientos funcionales.

Funcion	*Puntaje*
CLASES Y ESTRUCTURAS	*1*
MENÚ INTERACTIVO	*1*
SYSTEM	*1*
ADD-DRIVE	*1*
REGISTER	*1*
LOGIN	*1*
LOGOUT	*1*
SWITCH-DRIVE	*1*
MKDIR	*1*
CD	*1*
ADD-FILE	*1*
DEL	*1*
COPY	*1*
MOVE	*1*
RENAME	*0*
DIR	*0*
FORMAT	*0*
ENCRYPT	*0*
DECRYPT	*0*
GREP	*0*
VIEW-TRASH	*0*
RESTORE	*0*

Figura 1 menús del programa en consola de intelliJ, Izquierda menú inicial, derecha menú al seleccionar opcion 2 (modificar el sistema).

```
-----Escoja una opcion------
1. Crear Sistema
2. Modificar el Sistema
3. Ver el Sistema
4. Salir
ingrese su opción:
```

```
 --- Escoja una opción: ---
1.- Agregar un drive
2.- Crear un usuario
3.- Login usuario
4.- Desconectar un usuario
5.- Fijar un drive
6.- Crear un directorio:
7.- Cambiar de directorio
8. Agregar archivo
9. Eliminar archivo o carpeta
10. Copiar archivo en un Directorio
11. Mover archivo a un directorio
ingrese su elección:
```

Figura 2 Menú luego de elegir la opción 1 (crear sistema) para luego mostrar el resultado.

```
ingrese su opción:
1
Ingrese el nombre : hola
Se ha creado el sistema con éxito.
-----Escoja una opcion------
1. Crear Sistema
2. Modificar el Sistema
3. Ver el Sistema
4. Salir
ingrese su opción:
3
isidoraoyanedel21168603{nombre='hola'
 fechaCreacrion=Sun Jul 16 20:57:58 CLT 2023
 drives=[]
 usuarios=[]
 logueados=[]
 actual=[]
 ruta=[]
```

Figura 3 Consola de intelliJ luego de haber realizado 5 Modificaciones al sistema a través del menú.

```
isidoraoyanedel21168603{nombre='hola'
 fechaCreacrion=Sun Jul 16 21:00:33 CLT 2023
 drives=[Drive{letter='C', nombre='drive1', capacity=400}]
 usuarios=[Usuario{nombre='Isidora'}]
 logueados=[Isidora]
 actual=[C]
 ruta=[Path{rutaSTRING='c:/', carpeta=[], archivo=[], usuariocarpeta='Isidora', fechaCreacion=Sun Jul 16 21:00:55 CLT 2023}
]
```

Figura 4 UML del programa PREDICCIÓN

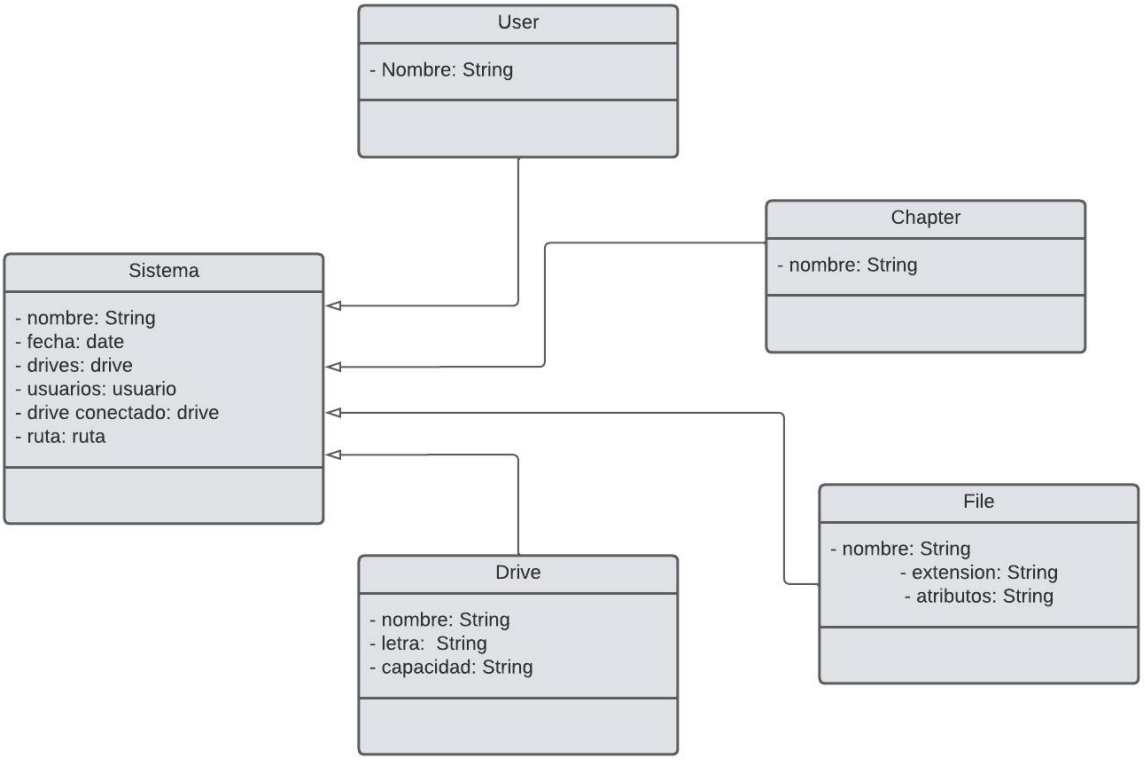
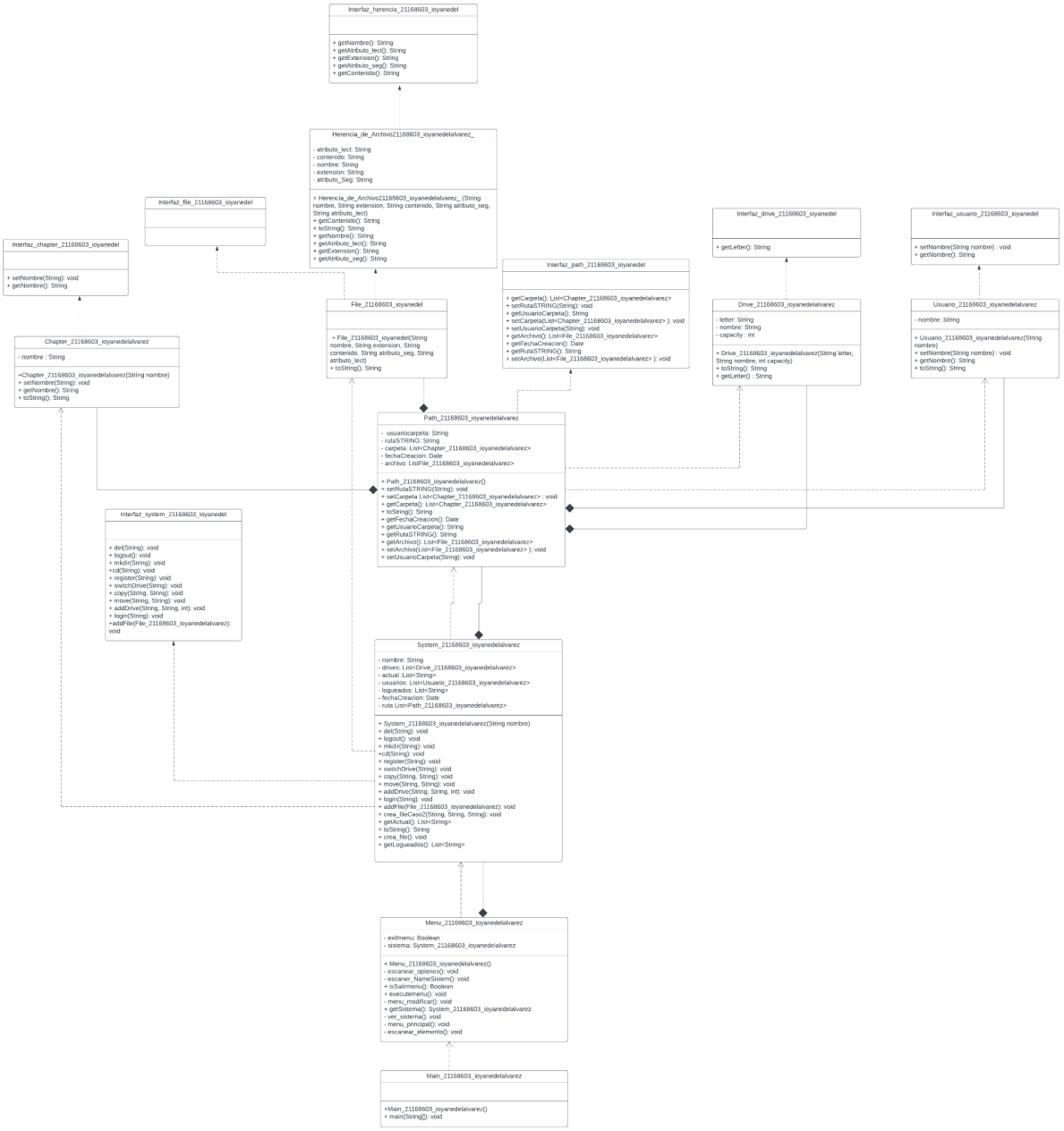


Figura 5 UML del programa FINAL



(ver en .zip o github)